

A SEMINAR REPORT ON

Building AI Agents for Autonomous Clouds: Challenges and Design Principles

Submitted By

ALEENA THOMAS (CEC22CS025)

under the esteemed guidance of

Mrs. RAKHI R

(Assistant Professor)

Department of Computer Engineering



SEPTEMBER 2025

**Department of Computer Engineering
College of Engineering, Pallippuram P O, Cherthala,
Alappuzha Pin: 688541,
Phone: 0478 2553416, Fax: 0478 2552714
<http://www.cectl.ac.in>**

A SEMINAR REPORT ON

Building AI Agents for Autonomous Clouds: Challenges and Design Principles

Submitted By

ALEENA THOMAS (CEC22CS025)

under the esteemed guidance of

Mrs. RAKHI R

(Assistant Professor)

*In partial fulfillment of the requirements for the award of the degree
of
Bachelor of Technology
in
Computer Science and Engineering
of
APJ Abdul Kalam Technological University*



SEPTEMBER 2025

**Department of Computer Engineering
College of Engineering, Pallippuram P O, Cherthala,
Alappuzha Pin: 688541,
Phone: 0478 2553416, Fax: 0478 2552714
<http://www.cectl.ac.in>**

DEPARTMENT OF COMPUTER ENGINEERING
COLLEGE OF ENGINEERING CHERTHALA
ALAPPUZHA-688541



C E R T I F I C A T E

This is to certify that, the seminar report titled ***Building AI Agents for Autonomous Clouds: Challenges and Design Principles*** is a bonafide record of the **CSQ413 SEMINAR** presented by **ALEENA THOMAS** (CEC22CS025) Seventh Semester B. Tech. Computer Science & Engineering student under our guidance and supervision in partial fulfillment of the requirements for the award of the degree **B. Tech. Computer Science & Engineering** of **APJ Abdul Kalam Technological University**.

Guide

Co-ordinator

HoD

Mrs. RAKHI R

Assistant Professor

Dept.of Computer Engg

Mrs. SHEMEEMA HASHIM Dr. PREETHA THERESA JOY

Assistant Professor

Dept. of Computer Engg

Professor

Dept. of Computer Engg

ACKNOWLEDGEMENT

This work would not have been possible without the support of many people. First and the foremost, I give thanks to Almighty God who gave me the inner strength, resource and ability to complete my seminar successfully.

I would like to thank **Dr. Jaya V L**, the Principal, who has provided with the best facilities for the seminar completion and presentation. I would also like to thank my HoD **Dr. PREETHA THERESA JOY** (Professor, Computer Engineering), my seminar coordinators **Mrs.SHEMEEMA HASHIM** (Assistant Professor, Computer Engineering) and **Mrs. Janu R. Panicker** (Assistant Professor, Computer Engineering) And my guide **Mrs. RAKHI R** (Assistant Professor, Computer Engineering) for the help extended and also for the encouragement and support given to me while doing the seminar.

I would like to thank my dear friends for extending their cooperation and encouragement throughout the seminar work, without which I would never have completed the seminar this well. Thank you all for your love and also for being very understanding.

ABSTRACT

The integration of Large Language Models (LLMs) and AI Agents marks a transformative advancement in cloud computing, particularly in the pursuit of secure, autonomous, and resilient cloud operations. While much focus has been placed on code generation, a more impactful application lies in enhancing the operational resilience of cloud services that today demand extensive human expertise and intervention.

By incorporating AI agents into cloud management, the project aims to improve fault detection, localization, and mitigation, thereby reducing service downtime and minimizing customer impact. The use of AI for IT Operations (AIOps) brings the vision of self-healing and autonomous clouds closer to reality. Through frameworks such as AIOpsLab, agents can interact with applications, inject faults using chaos engineering, and automatically diagnose and resolve failures. This not only increases system reliability but also reduces dependency on human operators.

Contents

1	Introduction	1
2	Fundamentals AI Agents and Autonomous Clouds	3
2.1	AI Agents	4
2.2	AIOps	4
2.3	AIOpsLab	5
2.4	Observability and Evaluation Frameworks	5
3	Literature Survey	6
3.1	Existing Research on AI Agents	6
3.2	Existing Research on Cloud Resilience	7
3.3	Integration of AI Agents and Autonomous Clouds	7
3.4	Existing Research on Fault Mitigation Lifecycle	8
4	AIOpsLab	9
4.1	Requirements and Principles	9
4.2	System Architecture	12
5	Design Decisions	14
5.1	Design Decisions	14
6	System Model	16
6.1	Prototype System Architecture	16

7	Case study and Insights	18
7.1	Case Study and Insights	18
7.1.1	Setup	18
7.1.2	Evaluation Goals	18
7.1.3	Key Insights	19
8	Motivation	21
9	Conclusion	22
9.1	Conclusion	22
10	References	23

List of Figures

4.1	SYSTEM ARCHITECTURE:	13
7.1	Figure 1: Orchestrator Figure 2: Agent	20

Chapter 1

Introduction

The integration of Artificial Intelligence (AI) Agents and Large Language Models (LLMs) into cloud computing represents a transformative step toward building autonomous and self-healing systems [3]. While AI and LLMs have already demonstrated remarkable progress in areas such as code generation, reasoning, and automation [3] their role in cloud operations is becoming increasingly critical. Modern cloud services, which form the backbone of enterprise and web-scale applications, demand high availability, fault tolerance, and rapid recovery [7]. However, ensuring operational resilience in such environments continues to require significant human expertise and intervention [31].

AI for IT Operations (AIOps) seeks to address these challenges by automating complex operational tasks such as fault detection, localization, root-cause analysis, and mitigation [5], [19], [20], [22], [44], [47]. The deployment of AI agents in this context holds the potential to not only reduce downtime but also minimize customer impact and improve system reliability [29], [31]. Unlike traditional monitoring and troubleshooting tools, AIOps agents can learn from real-time system data, interact with services, and take corrective actions with minimal human oversight [22], [44]. This capability brings the vision of autonomous and self-healing clouds closer to reality [24], [29].

Despite the promising benefits, the adoption of AI agents for cloud resilience introduces new challenges. Chief among these are the absence of standardized frameworks for building, evaluating, and improving AIOps agents [10] and the difficulty of reproducing realistic fault scenarios in a controlled yet reliable manner [1]. Without such frameworks, comparing different approaches and ensuring robustness in real-world deployments remains difficult [29].

This project builds upon these gaps by introducing AIOpsLab, a prototype system that provides an agent-cloud interface for orchestrating applications, injecting real-time faults through chaos engineering, and enabling AI agents to diagnose and resolve incidents [1]. By combining observability, workload generation, and automated fault injection [10] AIOpsLab lays the foundation for designing a modular and extensible framework to support the next generation of AI-driven cloud operations.

Traditional approaches to monitoring and fault management are often inadequate in handling the scale, dynamism, and unpredictability of these systems [29]. This makes the role of AI agents even more critical, as they can continuously adapt to evolving workloads, analyze diverse telemetry such as logs and traces, and execute corrective measures in near real-time [22], [36]. By embedding intelligence directly into the cloud management process, AI agents not only improve operational efficiency but also pave the way for scalable, resilient, and fully autonomous cloud ecosystems [24].

The introduction thus establishes the context for exploring the challenges and design principles of AI agents in autonomous clouds. It highlights the unique strengths of AI and cloud technologies, outlines their complementary role in ensuring resilience, and emphasizes the pressing need for standardized solutions to address emerging operational and security concerns [5].

Chapter 2

Fundamentals AI Agents and Autonomous Clouds

The integration of Artificial Intelligence (AI) Agents and Large Language Models (LLMs) into cloud computing marks a significant step toward creating autonomous, self-healing systems [3]. AI agents, designed to perform tasks that traditionally require human expertise such as reasoning, learning, and adaptive decision-making, are increasingly being applied in areas like IT operations, fault detection, and root-cause analysis [5],.

This project investigates the combined potential of AI agents and autonomous clouds, focusing primarily on operational resilience and fault management. The convergence of these technologies promises to reduce human intervention in cloud operations, enhance system reliability, and create self-healing frameworks capable of detecting, localizing, and resolving failures in real time [29]. However, this integration also presents challenges, including the lack of standardized frameworks for developing and benchmarking AIOps agents the complexity of reproducing realistic fault scenarios [18]and the need for robust mechanisms to evaluate and improve agent performance .

The research builds on the vision of AIOps, aiming to establish a structured foundation for secure and efficient autonomous cloud systems [29]. Through frameworks like AIOpsLab, the study explores agent-cloud interfaces, fault injection using chaos engineering, and automated resolution strategies [1]. By addressing the technical and methodological gaps, the project seeks to advance the design principles required to build resilient, intelligent, and trustworthy autonomous cloud ecosystems [25].

2.1 AI Agents

1. **Autonomous Decision-Making:** AI agents are designed to perform tasks without direct human involvement, such as identifying failures, rerouting services, and restarting applications in cloud environments.
2. **Learning and Adaptation:** By leveraging data from logs, traces, and performance metrics, AI agents continuously improve their ability to detect anomalies and predict failures.
3. **Interaction with Cloud Systems:** Agents act through an agent-cloud interface, allowing them to monitor applications, manage workloads, and execute recovery operations in real time.
4. **Role in AIOps:** AI agents form the backbone of AI for IT Operations (AIOps) by automating tasks like fault localization, root-cause analysis, and service restoration.

2.2 AIOps

1. **Anomaly Detection:** Using ML models to detect irregular patterns in telemetry data before they escalate into major failures.
2. **Automated Remediation:** Executing corrective actions (e.g., restarting nodes, scaling resources) without human intervention.
3. **Continuous Feedback:** Improving system resilience by learning from past incidents and adapting to evolving workloads.

2.3 AIOpsLab

AIOpsLab is a prototype framework developed to test and improve AI agents for autonomous clouds. Its core components are:

1. **Agent-Cloud Interface:** Facilitates seamless communication between agents and the cloud environment.
2. **Fault Injection via Chaos Engineering** Introduces controlled failures in real time to test system resilience and agent responses.
3. **Observability Tools:** Collects data (logs, traces, metrics) to evaluate how agents perform during incidents.

2.4 Observability and Evaluation Frameworks

1. **Comprehensive Monitoring:** Autonomous clouds generate large volumes of telemetry data such as logs, metrics, and traces. Observability frameworks allow AI agents to access this data for accurate fault detection and analysis.
2. **Benchmarking AI Agents:** Standardized evaluation frameworks are required to compare the performance of different AIOps agents under similar conditions.
3. **Trust and Transparency:** By recording and explaining the decisions made by AI agents, observability ensures accountability, which is essential for building trust in autonomous systems.

Chapter 3

Literature Survey

Research on the integration of Artificial Intelligence (AI) with cloud systems has steadily grown, driven by the need to enhance operational resilience and reduce human intervention. Several studies in the field of AI for IT Operations (AIOps) have explored methods for automating cloud management tasks such as anomaly detection, fault localization, and incident resolution.

3.1 Existing Research on AI Agents

Numerous Studies emphasize the potential of AI agents to automate complex decision-making tasks, reduce human intervention, and improve system resilience. Research highlights include:

1. **AI for IT Operations (AIOps):** Use of machine learning models for anomaly detection, root-cause analysis, and performance optimization in large-scale cloud systems.
2. **Autonomous Decision-Making:** Development of reinforcement learning and adaptive agents capable of making corrective actions in real time.
3. **Limitations in Standardization:** Despite advancements, research lacks unified frameworks for evaluating and benchmarking AI agents across different environments.

3.2 Existing Research on Cloud Resilience

Cloud research highlights the importance of fault tolerance, scalability, and observability in modern infrastructures. Key insights include:

1. **Cloud Observability:** Use of metrics, logs, and traces for system monitoring and fault diagnosis in distributed environments.
2. **Challenges in Reproducibility:** Difficulty in reproducing realistic failure scenarios consistently, which hinders comparative evaluation of resilience strategies

3.3 Integration of AI Agents and Autonomous Clouds

Recent studies explore the convergence of AI agents with cloud infrastructures to achieve greater automation and self-healing capabilities. Key points include:

1. **Operational Automation:** AI agents reduce the need for human oversight by automating incident detection, fault localization, and remediation.
2. **Framework Proposals:** Applying AI to improve the efficiency of blockchain operations, such as optimizing consensus algorithms and detecting anomalies.
3. **Challenges and Opportunities:** Addressing issues of scalability, trust, and reproducibility while unlocking opportunities for fully autonomous and resilient cloud ecosystems.

3.4 Existing Research on Fault Mitigation Lifecycle

1. **Detection:** Prior works focus on automated monitoring systems and anomaly detection techniques that raise alerts when irregular behavior is observed in cloud services [62]. These systems reduce reliance on manual reporting and provide early indicators of potential failures.
2. **Diagnosis:** Several studies address the challenge of root-cause analysis, highlighting the need for systematic investigation and communication between teams . Despite advancements, diagnosis remains one of the most time-consuming stages due to the complexity of distributed systems.
3. **Mitigation:** Existing work discusses strategies for service recovery, including automated restarts, rerouting, and configuration adjustments [5]. Research also emphasizes the value of documenting mitigation steps in Incident Management Systems (IcM) for postmortem learning and improving future responses.

Chapter 4

AIOpsLab

Here presents the guiding principles and design considerations for the AIOpsLab framework, which is envisioned to support robust evaluation and deployment of AI-driven operations agents. The framework is designed to enable modularity, flexibility, scalability, and reproducibility while addressing realistic operational challenges.

4.1 Requirements and Principles

- R1: Modular Design: The framework must support modular components for applications, workloads, and agents. Standardized interfaces enable seamless integration of new elements, allowing diverse use cases such as enterprise assessments, AI agent research, and red-team simulations of complex fault scenarios.
- R2: Flexible Interfaces: AIOpsLab should provide multiple interaction modes for humans, digital agents, and AI agents. Humans may use web interfaces for log review and command execution, digital agents require APIs for integration, and LLM-based agents need prompt-based interactions to request and receive responses.
- R3: Scalability: The framework must operate across a wide range of spatial scales (from single VMs to large clusters) and temporal scales (minutes to days), ensuring compatibility with both small-scale experiments and large enterprise deployments. This also allows for the detection and mitigation of slow-emerging or periodic faults.

- R4: Reproducibility: To ensure reliable and consistent evaluation, deployments must be automated and reproducible. The framework should provide default metrics such as mitigation time and accuracy, while also supporting user-defined metrics for more detailed analysis.
- R5: Versatility in Operating Environments: AIOpsLab must operate in diverse environments, from live production systems (for chaos engineering) to sandboxed or fully synthetic environments, allowing flexible experimentation with realistic and “what-if” scenarios.
- R6: Cross-Layer Fault Support: Fault injection should be possible across all system layers: hardware, network, operating system, middleware, applications, and external services. The framework must support both real-world inspired faults and synthetic faults to evaluate cascading effects in distributed systems.
- R7: Realistic Workload Conditions: Workloads should reflect domain-specific characteristics such as burstiness, interarrival times, and parallelism. Realistic workloads are essential for effective benchmarking and for assessing AI agent performance under conditions similar to real-world usage.
- R8: Coverage of the Operations Lifecycle: The framework should support all stages of incident management, including fault detection, root-cause analysis, and mitigation. Both proactive (predictive) and reactive strategies must be supported.
- R9: Observability: Adequate visibility into logs, telemetry, traces, KPIs, configuration files, and source code is necessary for effective fault detection, localization, and mitigation.
- R10: Control for Agent Operations: Agents must be able to perform corrective actions, such as modifying configurations, restarting services, or executing diagnostic test suites. Hooks should be available at various system layers to enable these operations.

AIOpsLab is designed as a modular and flexible framework to support the evaluation and deployment of AI-driven operations agents. It provides diverse interfaces for human, digital, and AI agents, ensuring seamless integration with applications, workloads, and fault models [1]. The framework is scalable across spatial and temporal dimensions, enabling experiments ranging from

single virtual machines (VMs) to large-scale clusters, and from short-term event-driven tests to long-running resilience studies [16] .

4.2 System Architecture

The AIOpsLab system architecture consists of several key components:

- **Orchestrator:** The central coordinator that manages interactions between all system elements. It serves as the Agent-Cloud Interface (ACI), assigning tasks to agents, deploying workloads, injecting faults, and monitoring the overall system state.
- **Workload Generator:** Responsible for creating realistic workloads with domain-specific characteristics such as burstiness, parallelism, and variable interarrival times. It provides diverse scenarios to evaluate agent performance under different operating conditions.
- **Fault Generator:** Introduces faults across multiple layers, including hardware, network, OS, middleware, and applications. It supports both real-world inspired and synthetic faults to simulate cascading failures and test mitigation strategies.
- **Applications / Services:** Deployed applications on which workloads are executed and faults are injected. These can range from synthetic systems to production-like environments, allowing flexibility in experimentation.
- **Observability Layer:** Collects telemetry, metrics, logs, traces, and KPIs from deployed services. This data provides agents and researchers with the necessary visibility to detect, localize, and mitigate faults.
- **Agents:** Human, digital, or AI-driven entities that interact with the Orchestrator to solve tasks. They analyze the system state, execute corrective actions, and provide solutions, leveraging the provided interfaces and metrics.
- **Evaluation & Metrics:** The Orchestrator evaluates agent performance using predefined metrics such as accuracy, time to mitigate, and custom user-defined measures. This ensures reproducibility and standardization across experiments.

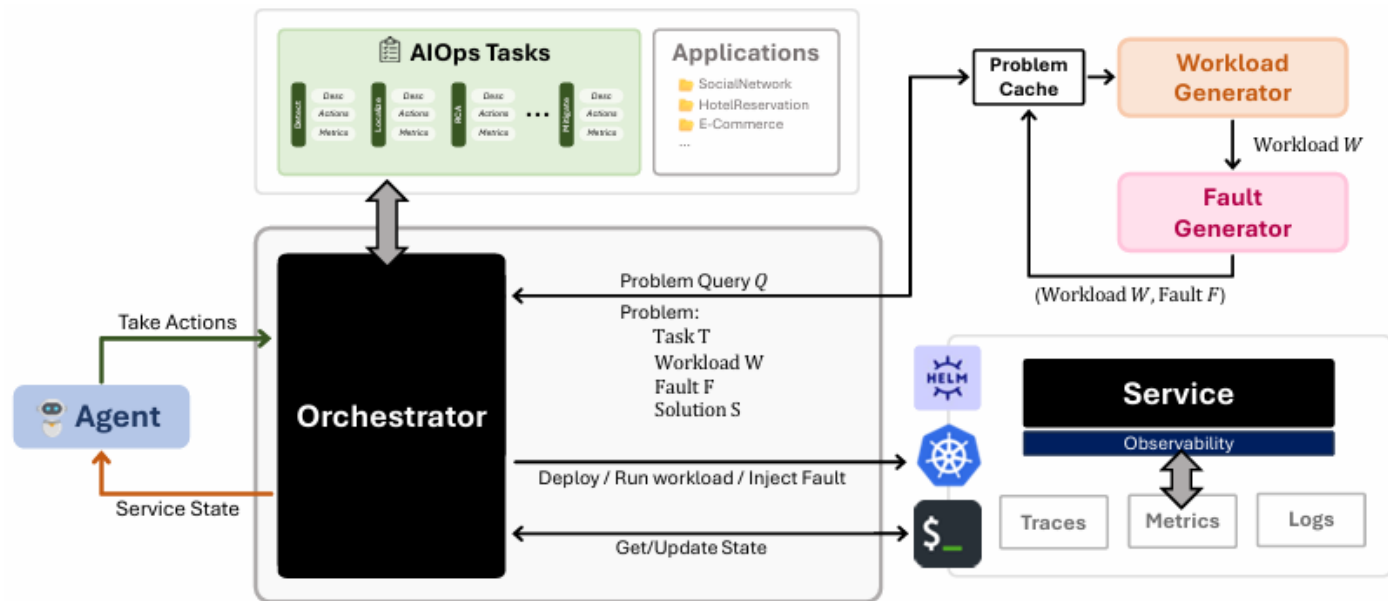


Fig. 4.1: SYSTEM ARCHITECTURE:

Chapter 5

Design Decisions

5.1 Design Decisions

The design of AIOpsLab is guided by requirements (R1–R10) and translated into a set of practical design decisions that shape the framework. These choices emphasize realism, scalability, observability, and flexibility to make the benchmark relevant for both research and practice.

- **Bring Your Services Alive (R1, R3, R4, R5):** We evaluate using live services designed with diverse architectures and at different scales. These services can operate in sandboxed, production-like, or real production environments, ensuring realistic fault and workload conditions. Automation tools such as Helm and Blueprint are adopted for repeatable and consistent setups.
- **Real Faults, Real Challenges (R6, R7):** Dynamic workload and fault generators simulate real-world conditions. They cover typical user behaviour, peak traffic loads, and realistic fault scenarios such as kernel crashes, network delays, and configuration bugs. This ensures agents face realistic challenges and avoids contamination of training data for LLM-based agents.
- **Evaluate Across the Board (R8):** By combining live services with real fault injections, the framework supports evaluation of the full incident management lifecycle — from detection to resolution. Evaluation spans both quantitative metrics (performance, time, resource usage, cost) and qualitative dimensions, including human or LLM-as-a-Judge assessment of agent traces.

- **Orchestrate the Agent-Cloud-Interface (R2, R10):** The Orchestrator functions as the Agent-Cloud Interface (ACI), mediating between agents and the cloud environment. It simplifies the action space into well-documented APIs, executes actions on behalf of agents, and provides structured feedback (e.g., logs and error messages), ensuring reliable and secure interaction.
- **Abstract Environments, not Agents (R2):** The framework abstracts environments (applications, workloads, and faults) while leaving agents flexible. This allows both human operators and automated AIOps tools (including LLMs) to interact consistently using the same problem descriptions, APIs, and instructions.
- **Observe Everything, Everywhere, All at Once (R9):** An observability layer collects traces, logs, metrics, cluster state, and configurations to provide comprehensive visibility into system behaviour. To prevent overwhelming tools with excessive data, flexible APIs allow users to query only the specific information needed.

Summary: These design decisions ensure that AIOpsLab is: (1) *Realistic* – reflecting operational challenges with live workloads and faults; (2) *Scalable* – supporting problem generation across different scales; (3) *Reliable* – covering the full lifecycle of fault management; (4) *Observable* – offering rich telemetry and state information; (5) *Flexible* – via the Agent-Cloud-Interface for diverse agent integration.

Chapter 6

System Model

6.1 Prototype System Architecture

This section expands on how the design decisions discussed in Section 3.2 lead to a prototype system architecture for AIOpsLab. The architecture consists of five key components, each playing a crucial role in enabling realistic evaluation of AIOps tools and agents.

- **Orchestrator:** The Orchestrator acts as an intermediate layer that separates the Agent from the Application Service. It manages sessions with agents by sharing problem descriptions, instructions, and a set of documented APIs (e.g., `get_logs`, `get_metrics`, `exec_shell`) to solve tasks. The Orchestrator validates and executes agent actions on the service’s behalf, with privileged access to tools such as `helm` and `kubectl`. It also coordinates the workload and fault generators to create live benchmark problems, and provides APIs for extensibility.
- **Service:** AIOpsLab abstracts a diverse set of application services to reflect the variance in production environments. These services include monolithic, microservice, and serverless architectures. Open-source benchmarks such as DeathStarBench are leveraged, along with tools like Blueprint, to enable deployment of realistic academic and production-scale services.
- **Workload Generator:** The workload generator simulates both normal and faulty operational scenarios. It receives specifications from the Orchestrator, including task, scale, and

duration, and can replay models trained on real production traces. Workloads capture patterns such as daily user activity cycles or cascading failures. Multiple workloads can be cached for reuse, and when combined with the Fault Generator, they produce complex fault scenarios.

- **Fault Generator:** The fault generator enables fine-grained fault injection across different system levels. It incorporates application and domain knowledge to simulate realistic incidents such as kernel failures, misconfigurations, or cascading microservice failures. This
vers

Chapter 7

Case study and Insights

7.1 Case Study and Insights

As a proof-of-concept, we implement a prototype of AIOpsLab and evaluate an LLM-based agent on an AIOps incident mitigation task. This case study demonstrates the potential of AIOpsLab to standardize evaluation for AIOps tools and uncover novel insights.

7.1.1 Setup

We deploy the *SocialNetwork* application from DeathStarBench on a Kubernetes cluster [27]. To emulate realistic failure scenarios, a misconfiguration fault is introduced at the virtualization layer, where the target port of a microservice is incorrectly set, leading to connectivity issues between dependent services [19]. Traffic is generated using an exponential workload pattern with the `wrk` benchmarking tool [33]. We then evaluate a ReAct agent powered by GPT-4, which leverages interleaved reasoning and action mechanisms to diagnose and mitigate the fault [12]. *Thought-Action* traces to plan and execute remediation steps.

7.1.2 Evaluation Goals

The case study highlights several requirements outlined in Section 3.1:

- **Reproducibility (R4):** Quantitative metrics include Success, Time-to-Detect (TTD), Time-to-Mitigate (TTM), and Efficiency.
- **Lifecycle Coverage (R8):** The experiment covers detection, root-cause analysis, and mitigation stages.
- **Observability (R9):** Logs, metrics, and traces are exposed to the agent.
- **Agent Control (R10):** The agent interacts through well-defined APIs, including `get_logs`, `get_metrics`, `get_traces`, and `exec_shell`.

The agent successfully detected the problem in **14 seconds (TTD)** and mitigated it in **36 seconds (TTM)**. On average, it required 10–12 interactions, costing around \$0.25 per run.

7.1.3 Key Insights

From this case study, we derive several insights:

1. **Importance of Observability:** The agent’s ability to detect the misconfiguration fault depended heavily on detailed telemetry. For example, repeated connection refusals in the logs guided its hypothesis about the root cause.
2. **Efficiency of Actions:** The design of APIs influences performance. Too few actions restrict exploration, while overly complex APIs hinder efficiency. Flexible APIs (e.g., `exec_shell`) enable agents to balance exploration and exploitation.
3. **Fault Injection Mirrors Real-World Complexity:** Even simple faults, such as Kubernetes misconfigurations, posed non-trivial challenges for GPT-4, requiring multiple interaction rounds. Automated fault generators thus provide rigorous testbeds for AIOps evaluation.
4. **Service Dependency Awareness:** Logs alone allowed the agent to infer some service dependencies, but for complex systems this can be insufficient. Future research may benefit from explicit dependency analysis tools to augment AIOps agents.
5. **Value of Robust Error Handling:** The agent improved its performance by quickly



Fig. 7.1: Figure 1: Orchestrator Figure 2: Agent

recovering from errors (e.g., correcting a faulty command). High-quality error propagation and meaningful feedback are essential for effective agent operation.

Chapter 8

Motivation

Modern cloud systems operate at unprecedented scale and complexity, supporting millions of users through distributed services that span virtual machines, containers, and microservices [1]]. This scale introduces a wide range of potential failures, including hardware breakdowns, network delays, software bugs, and misconfigurations [18]. Managing these failures is a critical challenge for system operators, as even minor disruptions can cascade across interdependent services and impact large user populations [10].

AIOps (Artificial Intelligence for IT Operations) has emerged as a promising paradigm to automate tasks such as fault detection, root-cause analysis, and mitigation. However, current approaches face significant limitations:

- Lack of standardized benchmarks for evaluating AIOps tools and agents, making it difficult to compare results across research studies.
- Limited realism in existing testbeds, which often rely on synthetic workloads or narrow fault models, reducing their applicability to production-scale environments.
- Insufficient coverage of the operations lifecycle, with many tools focusing only on detection rather than end-to-end fault resolution.
- Barriers to reproducibility, as experiments are often difficult to replicate across different environments.

Chapter 9

Conclusion

9.1 Conclusion

In this work, we presented AIOpsLab, a principled framework for evaluating and advancing AIOps agents. The framework is grounded in well-defined requirements (R1–R10) and design decisions that emphasize modularity, scalability, reproducibility, observability, and realistic operating conditions [5]. By integrating live services, workload and fault generators, observability layers, and an Agent-Cloud Interface (ACI), AIOpsLab enables controlled yet realistic experimentation across the entire incident management lifecycle [1].

Through a prototype implementation and case study with an LLM-based ReAct agent [3] we demonstrated the feasibility and utility of the framework. The results highlighted the importance of detailed observability, efficient API design, and realistic fault injection in shaping agent performance [22], [29]. These insights confirm the need for rigorous, reproducible benchmarks to evaluate emerging AIOps tools [5].

Chapter 10

References

- [1] 2022. AWS Fault Injection Simulator. <https://aws.amazon.com/fis/>.
- [2] 2022. Jepsen. <https://jepsen.io/>.
- [3] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. GPT-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [4] AgentOps-AI. 2024. agentops. <https://github.com/AgentOps-AI/agentops>.
- [5] Toufique Ahmed, Supriyo Ghosh, Chetan Bansal, Thomas Zimmermann, Xuchao Zhang, and Saravan Rajmohan. 2023. Recommending Root-Cause and Mitigation Steps for Cloud Incidents using Large Language Models. In *Proceedings of the 45th International Conference on Software Engineering (ICSE'23)*.
- [6] Aider. 2024. How aider scored SOTA 26.3% on SWE Bench Lite. <https://aider.chat/2024/05/22/swe-bench-lite.html>.
- [7] Ramnatthan Alagappan, Aishwarya Ganesan, Yuvraj Patel, Thanumalayan Sankaranarayana Pillai, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. 2016. Correlated Crash Vulnerabilities. In *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation (OSDI'16)*.
- [8] Ahmed Alquraan, Hatem Takturi, Mohammed Alfatafta, and Samer Al-Kiswani. 2018. An Analysis of Network-Partitioning Failures in Cloud Systems. In *Proceedings of the 13th USENIX Conference on Operating Systems Design and Implementation (OSDI'18)*.

- [9] George Amvrosiadis, Jun Woo Park, Gregory R. Ganger, Garth A. Gibson, Elisabeth Baseman, and Nathan DeBardeleben. 2018. On the diversity of cluster workloads and its impact on research results. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*, 533–546.
- [10] Vaastav Anand, Deepak Garg, Antoine Kaufmann, and Jonathan Mace. 2023. Blueprint: A Toolchain for Highly-Reconfigurable Microservice Applications. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*, 482–497.
- [11] Shrey Baheti, Shreyas Badiger, and Yogesh Simmhan. 2021. VioLET: An Emulation Environment for Validating IoT Deployments at Large Scales. *ACM Trans. Cyber-Phys. Syst.*, 5(3), Article 25, 39 pages.
- [12] Radu Banabici and George Candea. 2012. Fast Black-Box Testing of System Recovery Code. In *Proceedings of the 7th European Conference on Computer Systems (EuroSys'12)*.
- [13] Chetan Bansal, Sundararajan Renganathan, Ashima Asudani, Olivier Midy, and Mathru Janakiraman. 2020. Decaf: Diagnosing and triaging performance issues in large-scale cloud services. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Software Engineering in Practice*.
- [14] Rodrigo N. Calheiros, Rajiv Ranjan, Anton Beloglazov, César A. F. De Rose, and Rajkumar Buyya. 2011. CloudSim: A toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms. *Software: Practice and Experience*, 41(1), 23–50.
- [15] Marco Canini, Daniele Venzano, Peter Perešini, Dejan Kostić, and Jennifer Rexford. 2012. A NICE Way to Test OpenFlow Applications. In *Proceedings of the 9th USENIX Conference on Networked Systems Design and Implementation (NSDI'12)*.
- [16] Chaos Mesh Community. [n.d.]. ChaosMesh. <https://chaos-mesh.org/>. Accessed: 2024-07-08.
- [17] ChaosBlade Team. [n.d.]. ChaosBlade. <https://github.com/chaosblade-io/chaosblade>. Accessed: 2024-07-08.
- [18] Haicheng Chen, Wensheng Dou, Dong Wang, and Feng Qin. 2020. CoFI:

- Consistency-Guided Fault Injection for Cloud Systems. In *Proceedings of the 35th ACM/IEEE International Conference on Automated Software Engineering (ASE'20)*.
- [19] Junjie Chen, Xiaoting He, Qingwei Lin, Yong Xu, Hongyu Zhang, Dan Hao, Feng Gao, Zhangwei Xu, Yingnong Dang, and Dongmei Zhang. 2019. An empirical investigation of incident triage for online service systems. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP'19)*.
- [20] Junjie Chen, Xiaoting He, Qingwei Lin, Hongyu Zhang, Dan Hao, Feng Gao, Zhangwei Xu, Yingnong Dang, and Dongmei Zhang. 2019. Continuous incident triage for large-scale online service systems. In *2019 34th IEEE/ACM International Conference on Automated Software Engineering (ASE'19)*.
- [21] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
- [22] Yinfang Chen, Huaibing Xie, Minghua Ma, Yu Kang, Xin Gao, Liu Shi, Yunjie Cao, Xuedong Gao, Hao Fan, Ming Wen, Jun Zeng, Supriyo Ghosh, Xuchao Zhang, Chaoyun Zhang, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Tianyin Xu. 2024. Automatic Root Cause Analysis via Large Language Models for Cloud Incidents. In *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys'24)*, Athens, Greece, April 22–25, 2024. ACM, 674–688.
<https://doi.org/10.1145/3627703.3629553>
- [23] Maria Christakis, Patrick Emmisberger, Patrice Godefroid, and Peter Müller. 2017. A General Framework for Dynamic Stub Injection. In *Proceedings of the 39th International Conference on Software Engineering (ICSE'17)*.
- [24] Yuanshun Dai, Yanping Xiang, and Gewei Zhang. 2009. Self-healing and Hybrid Diagnosis in Cloud Computing. In *Cloud Computing*, Martin Gilje Jaatun, Gansen Zhao, and Chunming Rong (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg.
- [25] Andrea Detti, Ludovico Funari, and Luca Petrucci. 2023. μ Bench: An Open Source Factory of Benchmark Microservice Applications. *IEEE Transactions on Parallel and*

- Distributed Systems* 34, 3 (2023), 968–980. <https://doi.org/10.1109/TPDS.2023.3236447>
- [26] Envoy Docs. 2022. Envoy Fault Injection. <https://www.envoyproxy.io/docs/envoy/latest/api-v3/extensions/filters/http/fault/v3/fault.proto>.
- [27] Michael Ferdman, Almutaz Adileh, Onur Kocerber, Stavros Volos, Mohammad Alisafae, Djordje Jevdjic, Cansu Kaynak, Adrian Daniel Popescu, Anastasia Ailamaki, and Babak Falsafi. 2012. Clearing the clouds: a study of emerging scale-out workloads on modern hardware. *ACM SIGPLAN Notices* 47, 4 (2012), 37–48.
- [28] Yu Gan, Yanqi Zhang, Dailun Cheng, Ankitha Shetty, Priyal Rathi, Nayan Katarki, Ariana Bruno, Justin Hu, Brian Ritchken, Brendon Jackson, et al. 2019. An open-source benchmark suite for microservices and their hardware-software implications for cloud & edge systems. In *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS’19)*, 3–18.
- [29] Vaibhav Ganatra, Anjaly Parayil, Supriyo Ghosh, Yu Kang, Minghua Ma, Chetan Bansal, Suman Nath, and Jonathan Mace. 2023. Detection Is Better Than Cure: A Cloud Incidents Perspective. In *Proceedings of the 31st Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE’23)*.
- [30] Peter Garraghan, Renyu Yang, Zhenyu Wen, Alexander Romanovsky, Jie Xu, Rajkumar Buyya, and Rajiv Ranjan. 2018. Emergent Failures: Rethinking Cloud Reliability at Scale. *IEEE Cloud Computing* 5, 5 (2018), 12–21. <https://doi.org/10.1109/MCC.2018.053711662>
- [31] Supriyo Ghosh, Manish Shetty, Chetan Bansal, and Suman Nath. 2022. How to Fight Production Incidents? An Empirical Study on a Large-Scale Cloud Service. In *Proceedings of the 13th Symposium on Cloud Computing (SoCC’22)*, 126–141.
- [32] Haryadi S. Gunawi, Thanh Do, Pallavi Joshi, Peter Alvaro, Joseph M. Hellerstein, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau, Koushik Sen, and Dhruba Borthakur. 2011. Fate and Destini: A Framework for Cloud Recovery Testing. In *Proceedings of the 8th USENIX Symposium on Networked Systems Design and*

Implementation (NSDI'11).

[33] Songqiao Han, Xiyang Hu, Hailiang Huang, Minqi Jiang, and Yue Zhao. 2022.

Adbench: Anomaly detection benchmark. In *Advances in Neural Information Processing Systems (NeurIPS'22)*, 35, 32142–32159.

[34] Johann Hauswald, Michael A. Laurenzano, Yunqi Zhang, Cheng Li, Austin Rovinski, Arjun Khurana, Ronald G. Dreslinski, Trevor Mudge, Vinicius Petrucci, Lingjia Tang, et al. 2015. Sirius: An open end-to-end voice and vision personal assistant and its implications for future warehouse-scale computers. In *Proceedings of the 20th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS'15)*, 223–238.

[35] Shilin He, Botao Feng, Lique Li, Xu Zhang, Yu Kang, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. 2023. STEAM: Observability-Preserving Trace Sampling. In *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP'23)*, 1750–1761. <https://doi.org/10.1145/3611643.3613881>

[36] Shilin He, Xu Zhang, Pinjia He, Yong Xu, Lique Li, Yu Kang, Minghua Ma, Yining Wei, Yingnong Dang, Saravanakumar Rajmohan, et al. 2022. An Empirical Study of Log Analysis at Microsoft. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE'22)*.

[37] Helm. 2024. Helm: The Package Manager for Kubernetes. <https://helm.sh/>.

[38] Victor Heorhiadi, Shriram Rajagopalan, Hani Jamjoom, Michael K. Reiter, and Vyas Sekar. 2016. Gremlin: Systematic Resilience Testing of Microservices. In *Proceedings of the IEEE 36th International Conference on Distributed Computing Systems (ICDCS'16)*.

[39] Galen Hunt and Doug Brubacher. 1999. Detours: Binary Interception of Win32 Functions. In *Proceedings of the 3rd USENIX Windows NT Symposium*.

[40] Istio Docs. 2022. Istio Fault Injection.

<https://istio.io/latest/docs/tasks/traffic-management/fault-injection/>.